FINAL PROJECT REPORT · CYBERFOX SOCIETY

CyberFox — AI-Assisted Security Event Detection

A defensive prototype that validates security logs, ranks suspicious events and explains every alert it raises.

Sumyia Butt · Cybersecurity R&D Internship, CyberFox Society · October 2026 · github.com/sumyiabutt/ai-assisted-security-event-detection

OBJECTIVE

Small security teams collect more event data than they can review by hand, so real attacks stay buried inside the same flat log as routine traffic. CyberFox was built as a working prototype that turns a CSV of login, web and connection events into a short, ranked and explainable alert queue.

THE PROBLEM

- **Repeated failed logins** — five or more failures from one IP inside ten minutes, raised to Critical when a success follows the burst (T1110).
- **Port sweeps and request floods** — fifteen or more distinct ports, or sixty or more requests per minute, from a single source (T1046 / T1498).
- **Access outside business hours** — a successful login between 23:00 and 05:00 from an unusual context (T1078).

WHAT WAS DELIVERED

- **Row-level validation** — every rejected row keeps its line number and a plain-language reason.
- **Two detectors** — four window-based rules and an Isolation Forest, merged into one ranked queue.
- **Full workflow** — dashboard, searchable alert triage, CSV preview, analysis result and sample data.
- **Persistence and tests** — SQLite for runs, events, alerts and rejects; 46 pytest tests (17 on routes).

AT A GLANCE

2,000	events analysed
12	alerts raised (6 rule · 6 ML)
173	events flagged (8.65%)
46	automated tests passing

Dataset. Seeded generator (seed 42): 2,000 rows, 1–3 March 2026, 29 source IPs, 19 users, 117 malicious rows injected across four attack patterns.

Stack. Python 3.12, Flask, pandas, scikit-learn, SQLite. Packaged for Vercel as a single Python function.

FIVE-STAGE PIPELINE

01 Validate — Columns, timestamps, IPs, ports and ranges; rejects keep their line number.

02 Detect — Four sliding-window rules and an Isolation Forest score the same events.

03 Correlate — Merge alerts, flag events, assign severity and MITRE technique.

04 Store — SQLite keeps runs, events, alerts and rejected rows across restarts.

05 Present — Dashboard, alert triage and upload analysis in a responsive dark UI.



Dashboard. Security overview from the stored run: total and analysed events, suspicious events, alert count with severity split, a normal-versus-suspicious donut and events per hour.

DETECTION LOGIC

RULE	THRESHOLD	SEVERITY	MITRE
Brute force	5+ failed logins per IP in 10 min	High → Critical on success	`T1110`
Port scan	15+ distinct ports per IP	High	`T1046`
Request flood	60+ events per IP per minute	High	`T1498`
Off-hours login	Successful login 23:00–05:00	Medium	`T1078`

Behavioural model. The ML path scores each source IP, not each row: total events, failed logins, distinct ports, peak requests per minute and off-hours logins.

An **Isolation Forest** (200 trees, contamination 0.2, seed 42) produces a 0–100 anomaly score; 85+ is Critical, 70+ High, 55+ Medium. Each reason names the largest z-score feature — e.g. "25 distinct ports versus median 4 across 29 IPs". The `label` column is used only by the evaluation script.

RESULTS ON THE BUNDLED DATASET

2,000
rows, all valid

117
malicious rows (5.9%)

12
alerts · 4C / 4H / 3M / 1L

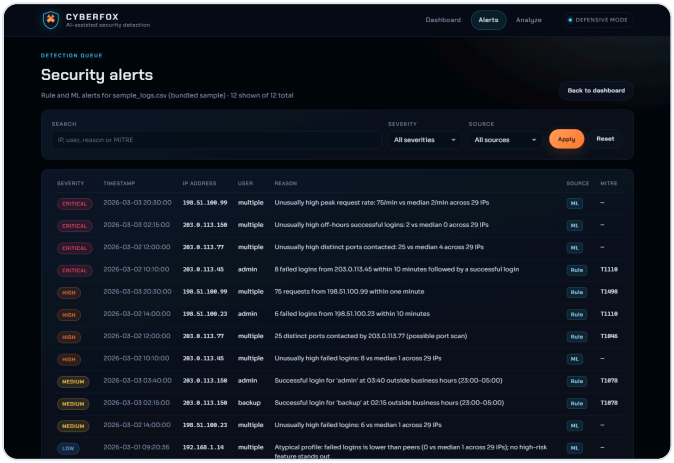
173
events flagged (8.65%)

APPROACH	TP	FP	FN	PRECISION	RECALL	F1
Rules	117	0	0	1.000	1.000	1.000
ML	117	56	0	0.676	1.000	0.807
Combined	117	56	0	0.676	1.000	0.807

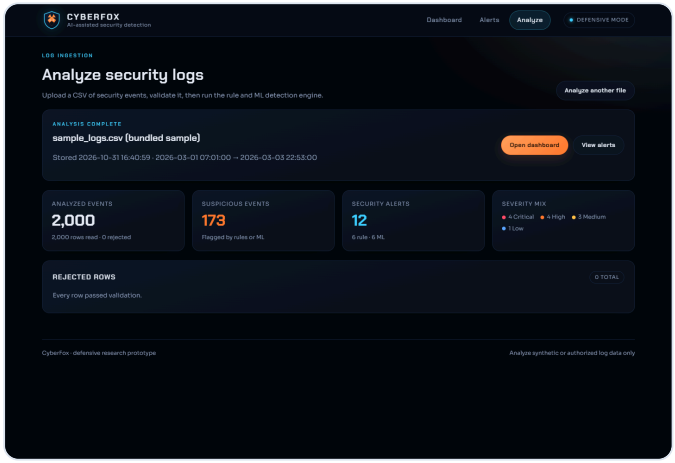
Produced by `python -m detection.evaluate` against the ground-truth label. Rules reach 1.000 because the synthetic attacks were generated to match their thresholds — this validates the implementation, not production accuracy.

Reading the numbers. The ML detector catches every malicious row through IP-level alerts, but also flags 56 benign events from one internal host (`192.168.1.14`) whose traffic profile is statistically unusual.

Combined equals ML because an event counts as detected if either approach covers it. It is honest recall at the cost of precision — the triage queue shows both, with the reason attached.



Alert triage. Severity, time, source IP, user, reason, detector and MITRE technique, with keyword search and severity/source filters.



Upload analysis. After preview, the run is stored and summarised with rejected-row reasons and links to the dashboard and alerts.

SECURITY CONTROLS

- CSV-only uploads checked after `secure_filename`; 10 MB request cap with a 413 page.
- Files stored under a random UUID token, validated, confined to the upload directory and deleted after analysis.
- Every SQL statement parameterised; all output rendered through Jinja autoescaping — no `eval`.

LIMITATIONS & NEXT STEPS

- One synthetic dataset, one contamination value: no train/test split or threshold tuning.
- No authentication, TLS or rate limiting yet — a local prototype, not a production service.
- Next: replay authorised real logs, tune thresholds with analyst feedback, and add an alert lifecycle.

Conclusion. CyberFox delivers an end-to-end defensive workflow — validation with line-level errors, four explainable rules, a deterministic anomaly model, a merged alert queue, SQLite persistence and a responsive dashboard. Every alert comes from the detection logic and every figure in this report from the project's own evaluation script and dataset.